

Task 4.5 – Malware Basics

Objective:

Study malware analysis concepts and analyze a benign executable (/usr/bin/ls) inside a Kali Linux virtual machine sandbox using static and dynamic analysis techniques.

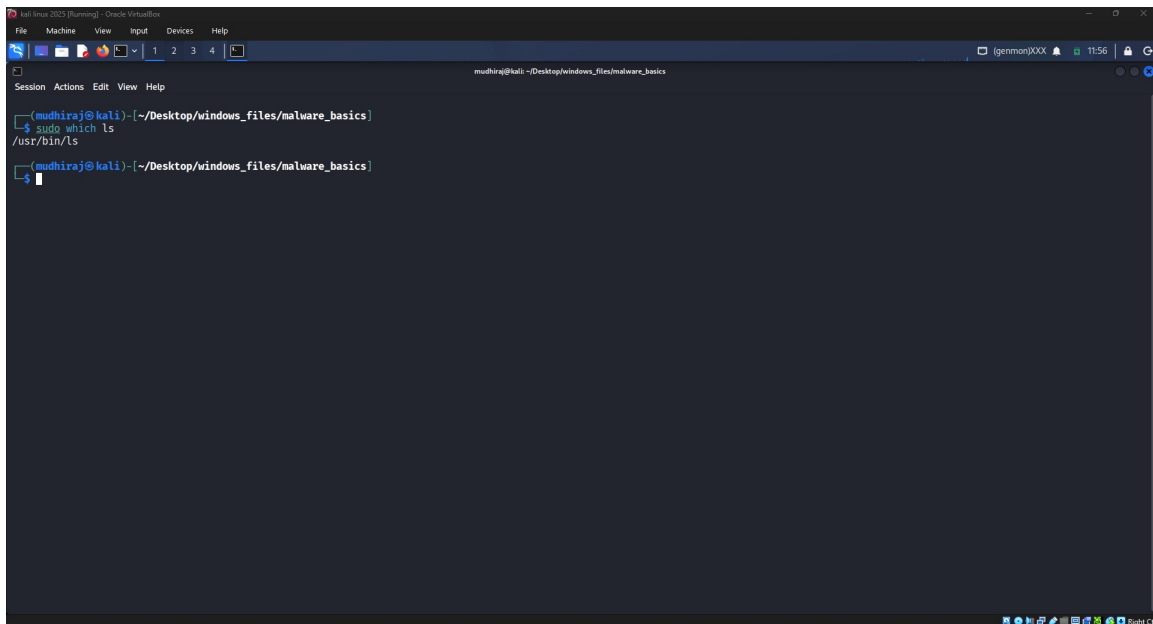
Environment:

- Host OS: Windows
- Guest OS: Kali Linux (VirtualBox)
- Sample: /usr/bin/ls (Benign Linux executable)
- Tools: which, file, strings, readelf, sha256sum, strace, ps

Analysis Steps and Observations:

Locate Sample:

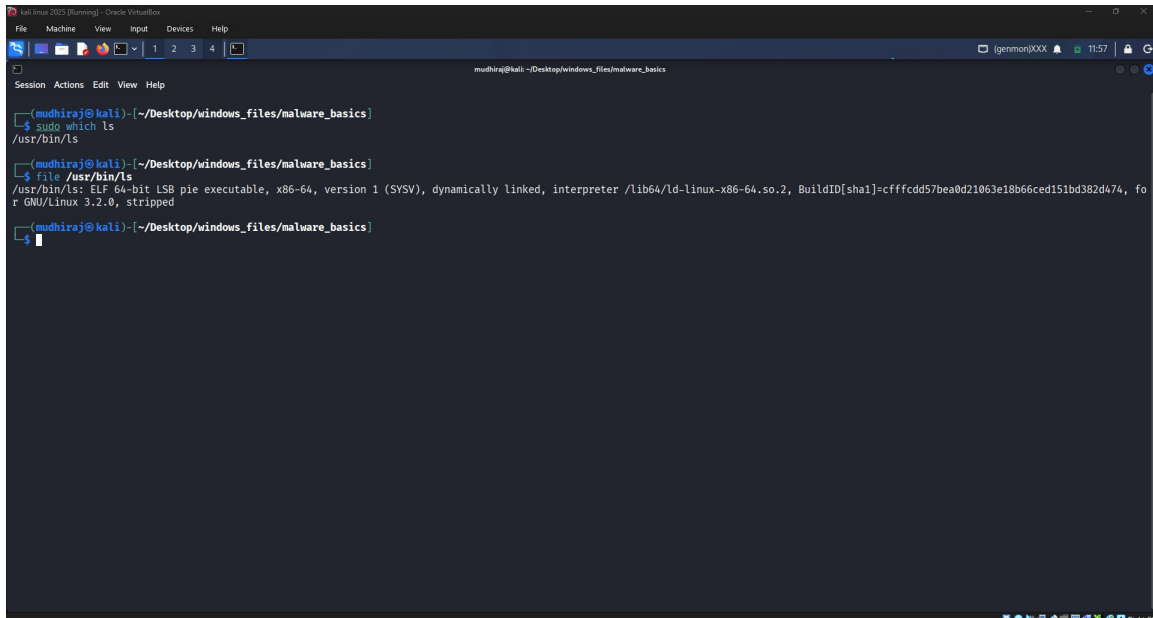
The 'which ls' command confirmed the executable location as /usr/bin/ls.



```
kali linux 2023 (Running) - Oracle VM VirtualBox
File Machine View Input Devices Help
1 2 3 4
mudhira@kali: ~/Desktop/windows_files/malware_basics
Session Actions Edit View Help
[mudhira@kali]~/Desktop/windows_files/malware_basics
$ sudo which ls
/usr/bin/ls
[mudhira@kali]~/Desktop/windows_files/malware_basics
$
```

File Identification:

The 'file' command identified it as a 64-bit ELF executable for Linux, dynamically linked and stripped.

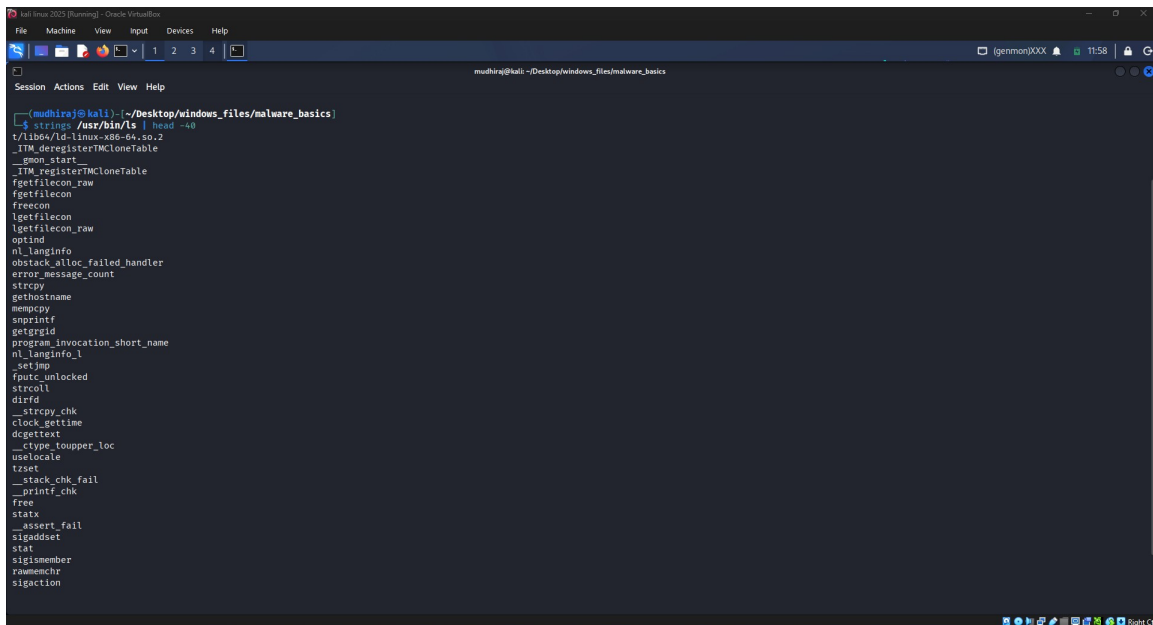


A screenshot of a Kali Linux terminal window. The prompt is (mudhira)@kali: ~/Desktop/windows_files/malware_basics. The user has run the command 'file /usr/bin/ls'. The output is: /usr/bin/ls: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=cffffdd57bea0d21063e18b66ced151bd382d474, for GNU/Linux 3.2.0, stripped.

```
(mudhira)@kali:~/Desktop/windows_files/malware_basics
$ sudo which ls
/usr/bin/ls
$ file /usr/bin/ls
/usr/bin/ls: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=cffffdd57bea0d21063e18b66ced151bd382d474, for GNU/Linux 3.2.0, stripped
```

Readable Strings:

The 'strings' command displayed embedded library names and readable symbols used by the executable.

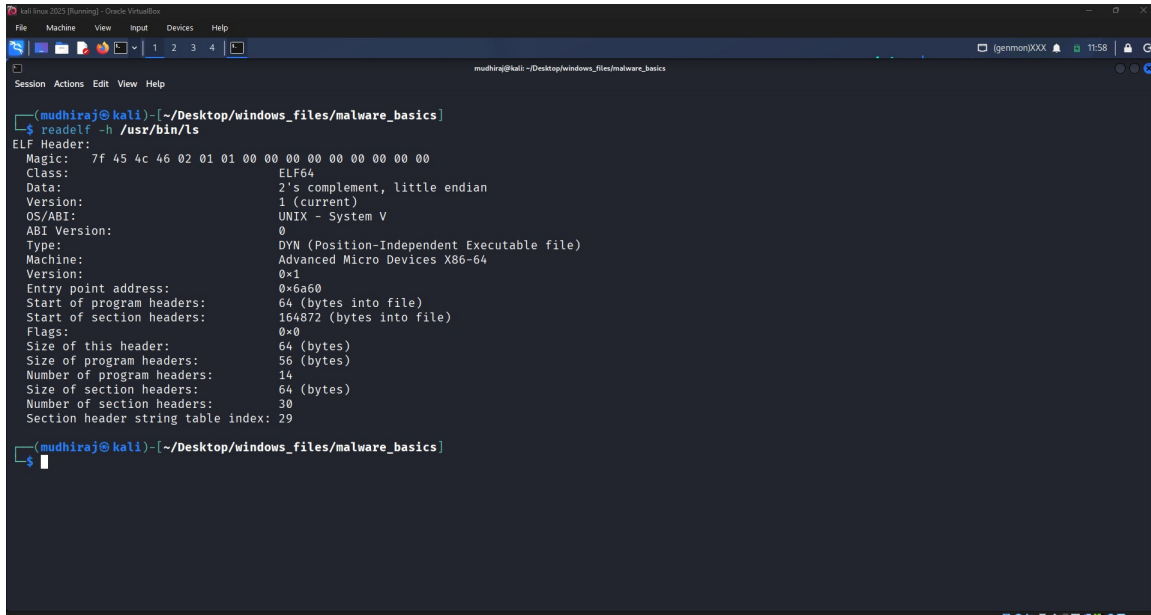


A screenshot of a Kali Linux terminal window. The prompt is (mudhira)@kali: ~/Desktop/windows_files/malware_basics. The user has run the command 'strings /usr/bin/ls | head -n 40'. The output lists various strings including library names like 't/lib64/ld-linux-x86-64.so.2' and symbols like '_ITM_deregisterTMCloneTable', '_dmon_start', 'fgetfilecon_raw', 'fgetfilecon', 'frecon', 'lgetfilecon', 'lgetfilecon_raw', 'optind', 'nl_langinfo', 'obstack_alloc_failed_handler', 'error_message_count', 'strcpy', 'gethostname', 'memcpy', 'snprintf', 'getrgid', 'program_invocation_short_name', 'nl_langinfo_l', '_setjmp', 'fputc_unlocked', 'strcoll', 'dirfd', 'strcpy_chk', 'clock_gettime', 'dcgettext', 'ctype_toupper_loc', 'uselocale', 'tzset', 'stack_chk_fail', 'printf_chk', 'free', 'stats', 'assert_fail', 'sigaddset', 'sigaction', 'stat', 'sigismember', 'rawmemchr', and 'sigaction'.

```
(mudhira)@kali:~/Desktop/windows_files/malware_basics
$ strings /usr/bin/ls | head -n 40
t/lib64/ld-linux-x86-64.so.2
_ITM_deregisterTMCloneTable
_dmon_start
_ITM_registerTMCloneTable
fgetfilecon_raw
fgetfilecon
frecon
lgetfilecon
lgetfilecon_raw
optind
nl_langinfo
obstack_alloc_failed_handler
error_message_count
strcpy
gethostname
memcpy
snprintf
getrgid
program_invocation_short_name
nl_langinfo_l
_setjmp
fputc_unlocked
strcoll
dirfd
strcpy_chk
clock_gettime
dcgettext
ctype_toupper_loc
uselocale
tzset
stack_chk_fail
printf_chk
free
stats
assert_fail
sigaddset
sigaction
stat
sigismember
rawmemchr
sigaction
```

ELF Header:

The 'readelf -h' command displayed the ELF header, architecture (x86-64), entry point, and section information.

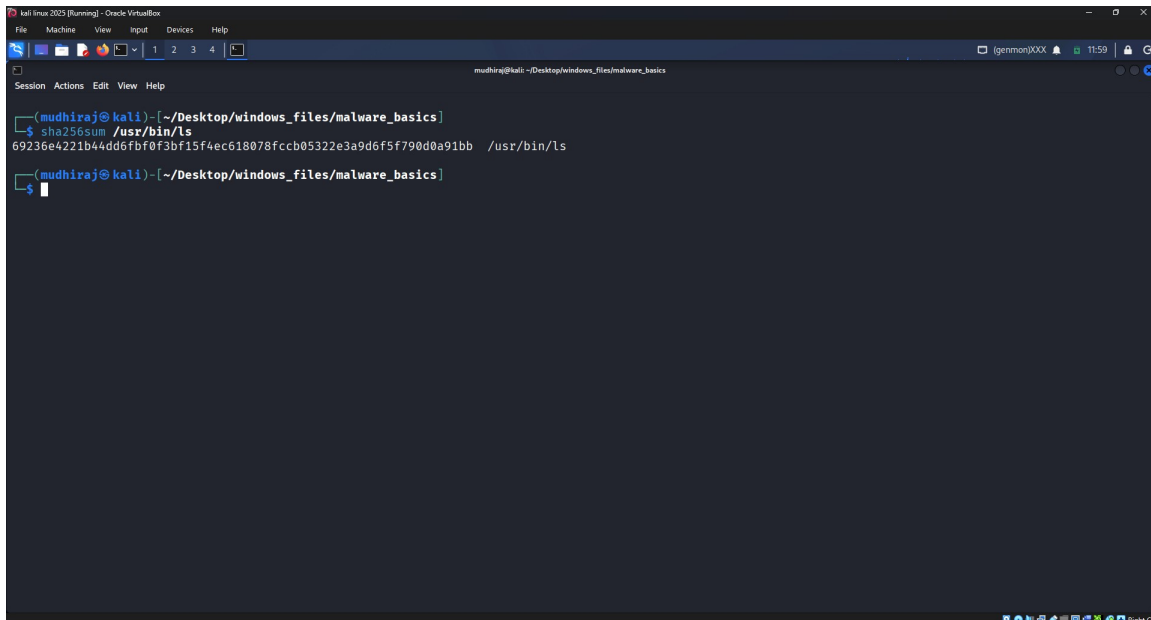


```
(mudhiraj@kali)-[~/Desktop/windows_files/malware_basics]
└─$ readelf -h /usr/bin/ls
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                               2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                             UNIX - System V
  ABI Version:                        0
  Type:                               DYN (Position-Independent Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:                 0x6a60
  Start of program headers:            64 (bytes into file)
  Start of section headers:           164872 (bytes into file)
  Flags:                               0x0
  Size of this header:                 64 (bytes)
  Size of program headers:             56 (bytes)
  Number of program headers:           14
  Size of section headers:             64 (bytes)
  Number of section headers:           30
  Section header string table index:  29

(mudhiraj@kali)-[~/Desktop/windows_files/malware_basics]
└─$
```

SHA-256 Hash:

A SHA-256 hash was generated to uniquely identify the file and verify integrity.



```
(mudhiraj@kali)-[~/Desktop/windows_files/malware_basics]
└─$ sha256sum /usr/bin/ls
69236e4221b44dd6fbf0f3bf15f4ec618078fccb05322e3a9d6f5f790d0a91bb /usr/bin/ls

(mudhiraj@kali)-[~/Desktop/windows_files/malware_basics]
└─$
```

Dynamic Analysis:

The 'strace' command showed system calls such as `execve()`, `openat()`, `mmap()`, `read()`, `close()`, indicating normal operating system interaction.

[illegible]

Process Observation:

The 'ps aux | grep ls' command verified process visibility after execution.

```

(mudhiraj@kali) [~/Desktop/windows_files/malware_basics]
$ ps aux | grep ls
mudhiraj 1595  0.0  0.1 99128 8976 ?        S+sl 11:46   0:00 /usr/bin/pipewire-pulse
mudhiraj 1887  0.0  0.6 1091880 38908 ?        S+  11:46   0:00 /usr/lib/x86_64-linux-gnu/xfce4/panel/wrapper-2.0 /usr/lib/x86_64-linux-gnu/xfce4/panel/plugins/libxfce4powermanager.so 18 25165840 power-manager-plugin Power Manager Plugin Display the battery level of your devices and control the brightness of your display
mudhiraj 5130  0.0  0.0 6556 2352 pts/0    S+   11:59   0:00 grep --color=auto ls

```

Security Assessment:

The analyzed sample is a legitimate Linux system utility and showed no suspicious behavior during analysis.

Observed behavior was limited to loading required libraries, reading configuration files, accessing directories, and interacting with the operating system through expected system calls.

No network communication, persistence mechanisms, privilege escalation, or malicious actions were observed.

Conclusion:

The benign executable `/usr/bin/ls` was successfully analyzed inside a sandbox environment using both static and dynamic analysis techniques. The executable exhibited expected behavior and no indicators of malicious activity were identified.